Setor de Educação Profissional e Tecnológica
Tecnologia em Análise e Desenvolvimento de Sistemas

DS122 - Desenvolvimento de Aplicações Web 1

11 - Back-end e PHP

Prof. Alex Kutzke

Outubro de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Explicar o que o interpretador do PHP faz com um arquivo `.php`, e o que chega ao navegador;
2. Iniciar o servidor embutido do PHP e abrir uma página por ele;
3. Declarar variáveis e usar `==` e `===`;
4. Montar textos com concatenação, interpolação e funções de string;
5. Criar arrays indexados e associativos, e percorrê-los com `foreach`;
6. Usar `if`, `elseif` e `else` para decidir que HTML é produzido;
7. Ler um arquivo JSON com `json_decode` e montar o catálogo a partir dele.

Roteiro

Back-end

Servidor

Sintaxe

Arrays

Controle

JSON

Uma página gerada no servidor

ola.php:

```php
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Olá, PHP</title>
</head>
<body>
  <h1>Olá, PHP</h1>
  <p>Número sorteado no servidor: <?= rand(1, 6) ?></p>
</body>
</html>
```

No terminal, na pasta do arquivo:

```
php -d display_errors=1 -S localhost:8000
```

e no navegador: `http://localhost:8000/ola.php`

O que chegou ao navegador

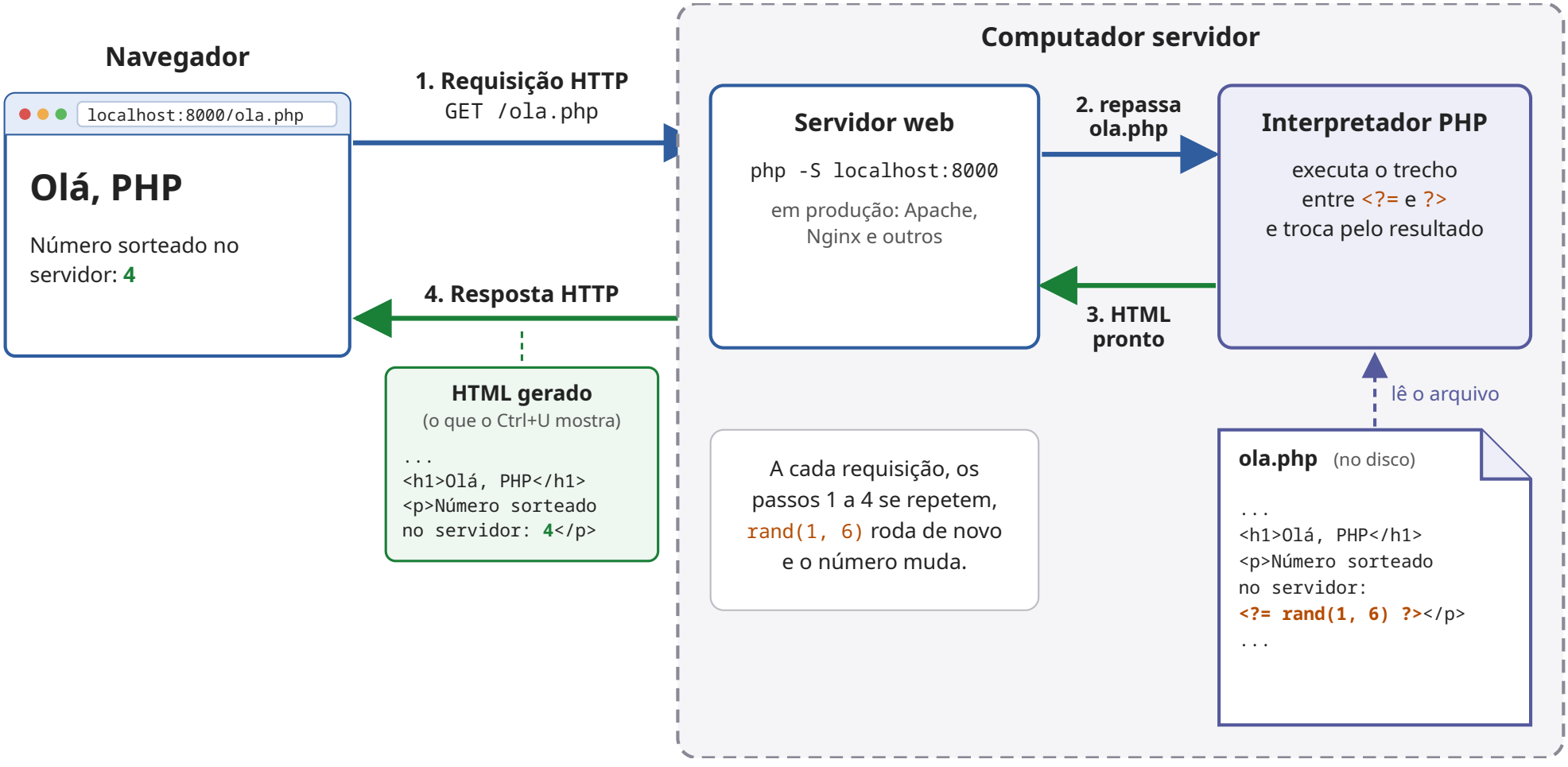
Cada recarga sorteia outro número.

Código-fonte da página, com `Ctrl+U`:

```
<body>
  <h1>Olá, PHP</h1>
  <p>Número sorteado no servidor: 4</p>
</body>
```

Nenhuma linha de PHP. O servidor passou o arquivo ao interpretador do PHP, que executou o trecho entre <?= e ?> e devolveu o HTML pronto. A resposta HTTP levou só o HTML.

Fluxo da requisição a `ola.php`



Back-end

Back-end (*server-side*): a parte da aplicação Web executada no servidor. Processa as regras de negócio, produz o HTML com conteúdo dinâmico e guarda os dados.

Para gerar páginas no servidor, três peças:

- um **servidor web**, que recebe as requisições HTTP e envia as respostas;
- um **interpretador** (ou programa), que executa a lógica e produz o HTML;
- um **banco de dados**, que guarda os dados da aplicação.

O banco de dados entra na disciplina na aula de PHP e MySQL.

Servidor web e diretório raiz

O servidor web escuta requisições, em geral na porta 80 (HTTP) e 443 (HTTPS). Exemplos: Apache, Nginx, Lighttpd.

Em páginas estáticas, ele traduz a URL para um arquivo dentro do **diretório raiz** (*document root*):

`http://example.com/dir1/arquivo.html`

`/var/www/html/dir1/arquivo.html`

O servidor nega acesso a arquivos fora do diretório raiz.

Páginas dinâmicas

Num sistema de notas, cada aluno vê as suas. Um arquivo HTML por aluno é inviável: o HTML precisa ser gerado a cada requisição.

1. o navegador pede `/catalogo.php`;
2. o servidor web vê o `.php` e passa o arquivo ao interpretador;
3. o interpretador executa o código e produz o HTML;
4. o servidor envia esse HTML na resposta.

Quem executa o código é o interpretador, não o servidor web. No Apache, por exemplo, a ligação é feita pelo módulo `mod_php` ou pelo serviço PHP-FPM.

Renderização no servidor, APIs e SPAs

Nesta parte da disciplina: **renderização no servidor** (SSR). O back-end monta o HTML completo de cada página.

Outro modelo, já visto pela metade com `fetch` e `produtos.json`:

- **API**: o back-end devolve apenas os dados, em geral em JSON, gerados a cada requisição;
- **SPA**: o front-end em JavaScript pede os dados com `fetch` e monta as telas pelo DOM, sem recarregar a página.

O segundo modelo é assunto de disciplinas seguintes do curso.

O servidor embutido do PHP

```
php -d display_errors=1 -S localhost:8000
```

- serve os arquivos da pasta onde o comando foi executado;
- `.php` passa pelo interpretador; HTML, CSS, imagens e JSON saem como estão;
- endereço sem arquivo: procura `index.php`, depois `index.html`;
- `Ctrl+C` encerra.

Feito para desenvolvimento. O manual do PHP avisa para não usá-lo em rede pública.

Instalação: `php-cli` pelo `apt` ou `dnf`; no Windows, o PHP do XAMPP. Ver o material de Programação Back-end.

Erros: página e terminal

`-d display_errors=1`: a mensagem de erro aparece na página. Sem a opção, conforme a instalação, a página chega em branco.

O terminal do servidor mostra uma linha por requisição, com status e erro:

```
[Mon Oct 26 12:35:47 2026] [::1]:33222 [200]: GET /ola.php
[Mon Oct 26 12:35:54 2026] [::1]:33250 [404]: GET /nada.php
                                    - No such file or directory
```

Live Server, `python3 -m http.server` e `file://` entregam o `.php` sem executar: o navegador mostra o código ou baixa o arquivo.

Prática: o ambiente

1. confira a instalação com `php -v`;
2. crie a pasta `aula-php/`, fora do repositório do trabalho;
3. crie nela o `ola.php`;
4. inicie o servidor dentro da pasta e abra `http://localhost:8000/ola.php`;
5. recarregue algumas vezes e abra o código-fonte com `Ctrl+U`.

Os exercícios desta aula não têm entrega.

Tempo: 10 minutos.

Resposta: o ambiente

```
$ php -v
PHP 8.3.x (cli) ...
```

- o número muda a cada recarga;
- o código-fonte mostra só o número, sem <?=;
- no terminal, uma linha `[200]: GET /ola.php` por recarga.

`php: command not found`: o PHP não está instalado, ou não está no `PATH`.

Tags PHP

O interpretador executa o que está entre as tags. O resto vai para a saída sem alteração.

- `<?php ... ?>`: bloco de código;
- `<?= ... ?>`: imprime um valor; equivale a `<?php echo ... ?>`.

```
<p>Isto vai direto ao navegador.</p>
<?php echo 'Isto é processado no servidor.'; ?>
<p>Isto também vai direto.</p>
```

O navegador recebe:

```
<p>Isto vai direto ao navegador.</p>
Isto é processado no servidor.<p>Isto também vai direto.</p>
```

A quebra de linha logo depois de ?> não vai para a saída.

Ponto e vírgula e a tag de fechamento

Cada instrução termina com ;.

Arquivo só com PHP, sem HTML: **sem** ?> no fim. Um espaço depois dele iria para a saída e causaria o erro *headers already sent*, visto nas aulas de formulários e de sessões.

```php
<?php
// Arquivo com apenas código PHP: termina sem a tag de
↪  fechamento.

echo 'Isto é um teste';
$variavel = 10;
```

Um ?> dentro de um comentário // também encerra o modo PHP. O que vem depois dele vira texto na página.

Variáveis e tipos

Toda variável começa com `$`. Sem `let` ou `const`: ela existe a partir da primeira atribuição. `$preco` e `$Preco` são variáveis diferentes.

```php
<?php
$nome = "Café em grão";    // string
$estoque = 12;             // int
$preco = 32.5;             // float
$disponivel = true;        // bool
$cupom = null;             // null

var_dump($preco);    // float(32.5)
var_dump($nome);     // string(14) "Café em grão"
```

14 é o tamanho em bytes: é e ã ocupam dois bytes em UTF-8.

Operadores e comparação

Aritméticos e de comparação iguais aos do JavaScript: `+ - * / %`, `< > <= >=`, `&& || !`.

```
var_dump(1 == "1");     // bool(true): converte o tipo
var_dump(1 === "1");    // bool(false): compara valor e tipo
```

Como no JavaScript, prefira `===` e `!==`.

Concatenação com ponto

O + é sempre soma. A concatenação é com `.`:

```
var_dump("1" + "1");   // int(2)
var_dump("1" . "1");   // string(2) "11"
```

`"Preço:  "+ $preco` interrompe o script:

```
Fatal error: Uncaught TypeError: Unsupported operand types: string +
↪  float
```

`.=` acrescenta ao fim:

```
$html = "<ul>";
$html .= "<li>Café em grão</li>";
$html .= "</ul>";
```

Aspas simples e aspas duplas

Aspas duplas substituem a variável pelo valor (**interpolação**), como a *template string* do JavaScript. Aspas simples não:

```
$nome = "Café em grão";
echo "Produto: $nome";       // Produto: Café em grão
echo 'Produto: $nome';       // Produto: $nome
echo "Produto: {$nome}!";    // Produto: Café em grão!
```

As chaves delimitam a variável. Use-as quando a variável é seguida de letra ou é um elemento de array.

Funções de string

Chamada	Resultado
`strlen("Chá")`	4 (bytes)
`strtoupper("Bebidas")`	`"BEBIDAS"`
`strtolower("Bebidas")`	`"bebidas"`
`trim("  Café  ")`	`"Café"`
`str_contains("Café em grão", "grão")`	`true`
`str_replace(".", ",", "32.5")`	`"32,5"`
`number_format(1234.5, 2, ",", ".")`	`"1.234,50"`

`number_format`: número, casas decimais, separador decimal, separador de milhar. Arredonda: `27.625` vira `"27,63"`.

`strtoupper("Café")` dá `"CAFé"`: só letras sem acento.

Prática: o preço com desconto

Crie o `preco.php`, com no topo:

```php
<?php
$nome = "Café em grão";
$categoria = "Bebidas";
$preco = 32.5;
$desconto = 0.15;
?>
```

1. calcule o preço com desconto em `$precoFinal`;
2. escreva um `<h1>` com o nome e a categoria em maiúsculas;
3. escreva um `<p>` com os dois preços, com `number_format`;
4. troque um `.` por `+` e leia o erro; desfaça.

Confira: `Café em grão (BEBIDAS)` e `De R$ 32,50 por R$ 27,63`.

Tempo: 10 minutos.

Resposta: o preço com desconto

```php
<?php
$nome = "Café em grão";
$categoria = "Bebidas";
$preco = 32.5;
$desconto = 0.15;
$precoFinal = $preco * (1 - $desconto);
?>
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Preço</title>
</head>
<body>
  <h1><?= $nome . " (" . strtoupper($categoria) . ")" ?></h1>
  <p>De R$ <?= number_format($preco, 2, ",", ".") ?>
     por R$ <?= number_format($precoFinal, 2, ",", ".") ?></p>
</body>
</html>
```

Arrays indexados

Valores em sequência, a partir do índice 0:

```
$categorias = ["Bebidas", "Doces", "Grãos"];
echo $categorias[0];         // Bebidas
echo count($categorias);     // 3

$categorias[] = "Temperos"; // acrescenta ao fim
echo count($categorias);     // 4
```

`$categorias[] = ...` faz o papel do `push` do JavaScript.

Arrays associativos

Cada valor tem uma **chave** de texto. => liga a chave ao valor:

```
$produto = [
  "id" => 1,
  "nome" => "Café em grão",
  "categoria" => "Bebidas",
  "preco" => 32.5,
  "desconto" => 0.15,
];

echo $produto["nome"];                  // Café em grão
echo "Preço: {$produto["preco"]}";      // Preço: 32.5
```

O papel do objeto no JavaScript, com acesso sempre por colchetes: `$produto["nome"]`, e não `produto.nome`.

Array de arrays

A forma do `produtos.js` e do `produtos.json`:

```
$produtos = [
  ["nome" => "Café em grão", "preco" => 32.5],
  ["nome" => "Chá mate", "preco" => 12.9],
];

echo $produtos[1]["nome"];   // Chá mate
print_r($produtos[0]);
```

```
Array
(
    [nome] => Café em grão
    [preco] => 32.5
)
```

`echo $produtos` imprime só `Array`, com o aviso `Array to string conversion`.

if, elseif e else

```
if ($produto["desconto"] > 0.2) {
  echo "Promoção";
} elseif ($produto["desconto"] > 0) {
  echo "Com desconto";
} else {
  echo "Preço normal";
}
```

elseif é uma palavra só.

foreach

Percorre um array, um elemento por vez:

```
foreach ($produtos as $produto) {
  echo "<li>{$produto["nome"]}</li>";
}
```

Com a chave:

```
foreach ($produto as $chave => $valor) {
  echo "$chave: $valor\n";
}
```

```
id: 1
nome: Café em grão
categoria: Bebidas
...
```

for e while

A forma do JavaScript, com $ nas variáveis:

```
for ($i = 1; $i <= 3; $i++) {
  echo "<p>Parcela $i</p>";
}

$estoque = 3;
while ($estoque > 0) {
  echo "<p>Restam $estoque</p>";
  $estoque--;
}
```

Sintaxe alternativa dentro do HTML

`:` no lugar de `{`, e `endif;`, `endforeach;` no lugar de `}`:

```
<ul>
  <?php foreach ($produtos as $produto): ?>
    <li>
      <?= $produto["nome"] ?>
      <?php if ($produto["preco"] < 20): ?>
        <strong>até R$ 20</strong>
      <?php endif; ?>
    </li>
  <?php endforeach; ?>
</ul>
```

O `<li>` se repete por produto; o `<strong>` só sai quando a condição é verdadeira. O HTML fica fora das strings.

Prática: o catálogo com `foreach`

Crie o `catalogo.php` com o array de três produtos da seção 14 do material (`id`, `nome`, `descricao`, `categoria`, `preco`, `desconto`).

1. escreva um `<p>` com a quantidade de produtos, com `count`;
2. escreva um `<article class="cartao»` por produto, com `foreach` na sintaxe alternativa: `<h2>` com o nome, `<p>` com a descrição, `<p>` com o preço;
3. confira três `<article>` no `Ctrl+U`;
4. acrescente um quarto produto ao array e confira que ele aparece.

Tempo: 15 minutos.

Resposta: o catálogo com `foreach`

```
<body>
  <p><?= count($produtos) ?> produtos</p>
  <?php foreach ($produtos as $produto): ?>
    <article class="cartao">
      <h2><?= $produto["nome"] ?></h2>
      <p><?= $produto["descricao"] ?></p>
      <p>R$ <?= number_format($produto["preco"], 2, ",", ".") ?></p>
    </article>
  <?php endforeach; ?>
</body>
```

O quarto produto aparece sem mudança no HTML: o `foreach` gera um cartão por elemento do array.

Funções

```
function formataPreco(float $valor): string {
  return "R$ " . number_format($valor, 2, ",", ".");
}

echo formataPreco(27.625);   // R$ 27,63
```

- tipos do parâmetro (`float $valor`) e do retorno (`: string`) são opcionais;
- com eles, `formataPreco("abc")` interrompe o script com `TypeError`;
- variáveis de fora não são visíveis dentro da função: o que ela usa entra pelos parâmetros.

Prática: o desconto com `if`

No `catalogo.php`:

1. escreva a função `formataPreco` no bloco do topo, e use-a em todos os preços;
2. troque o parágrafo do preço por um `if` na sintaxe alternativa: com desconto, o preço original dentro de `<s>` e o preço com desconto ao lado; sem desconto, só o preço.

Confira:

```
Café em grão      R$ 32,50 R$ 27,63
Chá mate          R$ 12,90
Pão de mel        R$ 8,00 R$ 7,20
```

Tempo: 15 minutos.

Resposta: o desconto com `if`

```php
<?php
function formataPreco(float $valor): string {
  return "R$ " . number_format($valor, 2, ",", ".");
}
// ... array $produtos ...
?>
<!-- dentro do foreach, no lugar do parágrafo do preço -->
<?php if ($produto["desconto"] > 0): ?>
  <p>
    <s><?= formataPreco($produto["preco"]) ?></s>
    <?= formataPreco($produto["preco"] * (1 - $produto["desconto"])) ?>
  </p>
<?php else: ?>
  <p><?= formataPreco($produto["preco"]) ?></p>
<?php endif; ?>
```

json_decode

O mesmo `produtos.json` lido pelo `fetch`, agora no servidor:

```
$conteudo = file_get_contents("produtos.json"); // texto
$produtos = json_decode($conteudo, true);         // array
```

- `true`: objetos do JSON viram arrays associativos;
- o resultado tem a forma do array de arrays, e o mesmo `foreach` serve;
- `json_encode` faz o caminho inverso, e é como um back-end PHP responde a um `fetch`.

JSON com erro

Com erro de sintaxe no arquivo, `json_decode` devolve `null`, sem mensagem:

```
$produtos = json_decode('[{"id": 1,}]', true);
var_dump($produtos);            // NULL
echo json_last_error_msg();     // Syntax error
```

O `foreach` logo depois avisa:

```
Warning: foreach() argument must be of type array|object, null given
```

Prática: o catálogo pelo `produtos.json`

1. copie para `aula-php/` o `produtos.json` da Entrega 2, ou escreva os três produtos em JSON;
2. abra `http://localhost:8000/produtos.json` e confira que o servidor entrega o arquivo;
3. no `catalogo.php`, troque o array escrito no código pela leitura do arquivo;
4. acrescente um produto ao JSON e confira que ele aparece sem alteração no PHP.

`Undefined array key "desconto"`: falta essa chave em algum produto do arquivo. Acrescente `"desconto": 0`, ou ajuste o nome no PHP.

Tempo: 10 minutos.

Resposta: o catálogo pelo `produtos.json`

```php
<?php
function formataPreco(float $valor): string {
  return "R$ " . number_format($valor, 2, ",", ".");
}

$conteudo = file_get_contents("produtos.json");
$produtos = json_decode($conteudo, true);
?>
```

O HTML abaixo do bloco não muda.

Na Entrega 3, o lugar do `json_decode` é ocupado pela consulta ao banco de dados.

Prática: filtro por categoria

No `catalogo.php`:

1. crie no topo a variável `$categoria`, com o valor `"Bebidas"`;
2. dentro do `foreach`, mostre apenas os produtos dessa categoria, com um `if`;
3. no topo, conte em `$mostrados` os produtos da categoria, com outro `foreach` e um `if`, e use a variável no `<p>` da quantidade;
4. troque o valor de `$categoria` e recarregue.

Tempo: 10 minutos.

Resposta: filtro por categoria

```php
<?php
// ... formataPreco e json_decode ...
$categoria = "Bebidas";
$mostrados = 0;
foreach ($produtos as $produto) {
  if ($produto["categoria"] === $categoria) {
    $mostrados++;
  }
}
?>
<!-- ... -->
<p><?= $mostrados ?> produtos</p>
<?php foreach ($produtos as $produto): ?>
  <?php if ($produto["categoria"] === $categoria): ?>
    <article class="cartao"> ... </article>
  <?php endif; ?>
<?php endforeach; ?>
```

Na aula de formulários, `$categoria` passa a vir do endereço, como em `catalogo.php?categoria=Bebidas`.

Quando dá errado

Sintoma	Causa provável
página mostra o código PHP, ou o navegador baixa o arquivo	sem o servidor do PHP
página em branco	`display_errors` desligado; erro no terminal
`Parse error: syntax error, unexpected token "echo"`	`;` faltando na linha anterior
`Undefined variable`	nome digitado diferente, ou variável de fora usada na função
`Undefined array key`	chave inexistente no elemento
`Unsupported operand types: string + float`	`+` no lugar de `.`
`Array` na página	`echo` de um array inteiro
`$nome` literal na página	aspas simples
`foreach() argument must be of type array|object, null given`	JSON com erro ou caminho errado

Exercícios e próximos passos

Os exercícios desta aula não têm entrega e não valem nota. O enunciado completo está na Parte prática do material de PHP.

Próxima aula: formulários. `$_GET` e `$_POST`, validação no servidor, `htmlspecialchars` e `include`.

A Entrega 3 do trabalho (PHP e MySQL) vence em 27/11, às 23h59. Não há entrega em atraso: vale o último *commit* enviado até o prazo.